Workshop 3: Deep Q-Learning - Final Report

Authors: Zhengnan Cao, Yuchen Yang

This report answers Task 1 and all three Task 2 questions using the audited experiment set. Each condition uses three training seeds (0, 1, 2), 200,000 training steps, and 10 greedy evaluation episodes per checkpoint. Evaluation occurs every 10,000 training steps using reset seeds 10000-10009. Evaluation steps are excluded from the horizontal axis. Curves show the mean and one sample standard deviation across training seeds, without smoothing.

Tables report mean ± SD. Last-three return averages three final checkpoints per seed. Threshold time confirms three consecutive evaluation means of at least 475; times are medians among successful seeds and counts show how many reached it. Confirmation does not guarantee later stability. Reference: provided ELEN90098 Workshop 3 handout. Three seeds give descriptive results, not a universal ranking.

Task 1 - Question 1: CartPole environment

Question

What are the state space, action space and reward structure of CartPole? Are they finite or infinite, discrete or continuous? Is the environment deterministic or stochastic?

Answer

Item	Answer
State	Four real-valued variables: cart position, cart velocity, pole angle and pole angular velocity. The state space is continuous and infinite.
Action	Two actions: 0 pushes left and 1 pushes right. The action space is discrete and finite.
Reward	+1 per step, including the first failure step. Immediate reward is discrete and finite. CartPole-v1 allows at most 500 steps, so the maximum episode return is 500.
Dynamics	Deterministic given a state and action. Reset adds random initial-state perturbations, so episodes can still differ.

The episode terminates when cart position leaves approximately ±2.4 m or pole angle leaves approximately ±12 degrees. The 500-step time limit truncates an otherwise continuing episode.

Task 1 - Question 2: DQN training

Question

Train DQN on CartPole-v1 and check whether it can achieve the maximum return of 500. Use TensorBoard, submit evaluation returns versus environment steps, and briefly comment on training success.

Submit

The required evaluation curve is shown below. This uses the learning-rate 0.001 runs from the corrected experiment set, rather than the historical Task 1 run.

Configuration: three hidden layers of 64 units, learning rate 0.001, discount 0.99, replay capacity 100,000, batch size 64, and target update every 500 gradient updates. Learning starts after 1,000 steps. Epsilon starts at 1, decays by 0.995 after each episode and has a minimum of 0.02.

The DQN target is:

$$y = r + \gamma(1 - d)\max_{a'} Q_{\text{target}}(s', a')$$

Here d is 1 for true environment termination and 0 otherwise; a time-limit truncation still permits bootstrapping.

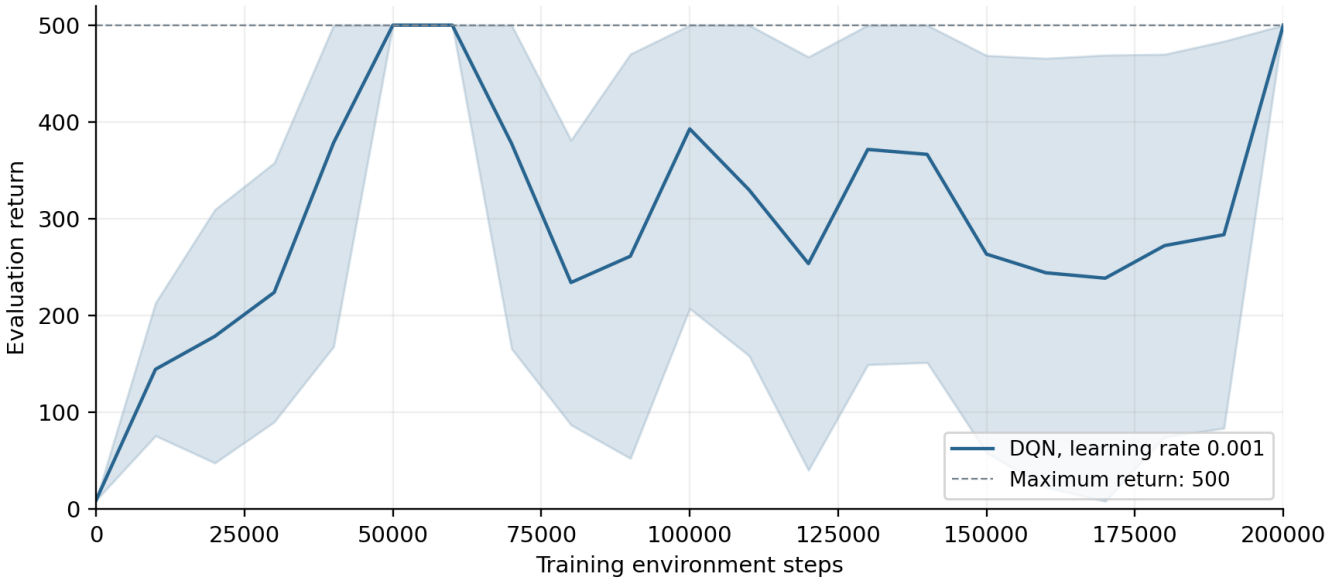


Figure 1. DQN evaluation returns versus training environment steps.

Discussion

Does the curve show successful training?

All three seeds finish with a 10-episode evaluation mean of 500, so the agent achieves the required maximum. However, the average of the last three checkpoints is 351.9 ± 82.1, showing earlier late-stage drops; this is successful maximum-return performance, not stable convergence throughout training.

TensorBoard scalar exports of these saved evaluations are provided in the separate verification material. The submitted curve uses the same evaluation records.

Task 2 - Question 1: Learning rate

Question

Investigate how learning rate affects training by comparing at least three values. Here the rates are 0.0001, 0.001 and 0.01, with all other settings fixed as above.

Submit

Submit evaluation returns versus environment steps for each learning rate.

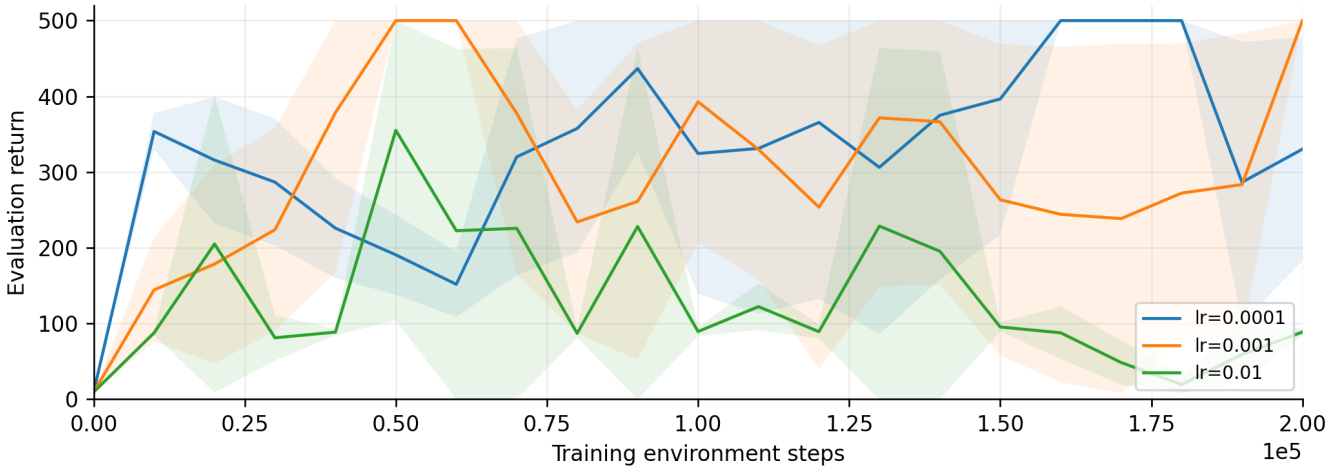


Figure 2. Learning-rate comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
0.0001	330.2 ± 147.6	372.2 ± 110.7	150,000 (3/3)
0.001	500.0 ± 0.0	351.9 ± 82.1	60,000 (3/3)
0.01	88.7 ± 4.5	55.7 ± 7.8	70,000 (1/3)

Discussion

Which learning rate achieves faster convergence?

Learning rate 0.001 confirms the threshold fastest across all three seeds: median 60,000 steps, compared with 150,000 for 0.0001. Rate 0.01 confirms in only one of three seeds.

Which learning rate is more stable?

Among the two useful rates, 0.0001 has smaller late-training fluctuations: mean temporal SD over the last five checkpoints is 95.0, versus 180.4 for 0.001. Its final return is lower, so stability and endpoint performance are different criteria.

Does too high a learning rate cause divergence or unstable oscillations?

At 0.01 the curves fluctuate and final return is only 88.7 ± 4.5. This indicates poor and unstable learning here; finite losses do not establish numerical divergence.

Does too low a learning rate cause slow learning or poor final performance?

Rate 0.0001 takes longer to confirm near-maximum performance and ends below 0.001. It still learns effectively and is more stable late in training, so a smaller rate is not simply ineffective.

Task 2 - Question 2: Exploration schedules

Question

Compare constant epsilon 0.1 with two episode-based decays: multiply epsilon by 0.9 or 0.995 after each completed episode, starting from 1 and stopping at a minimum of 0.02.

Submit

Submit evaluation returns versus environment steps for all three schedules.

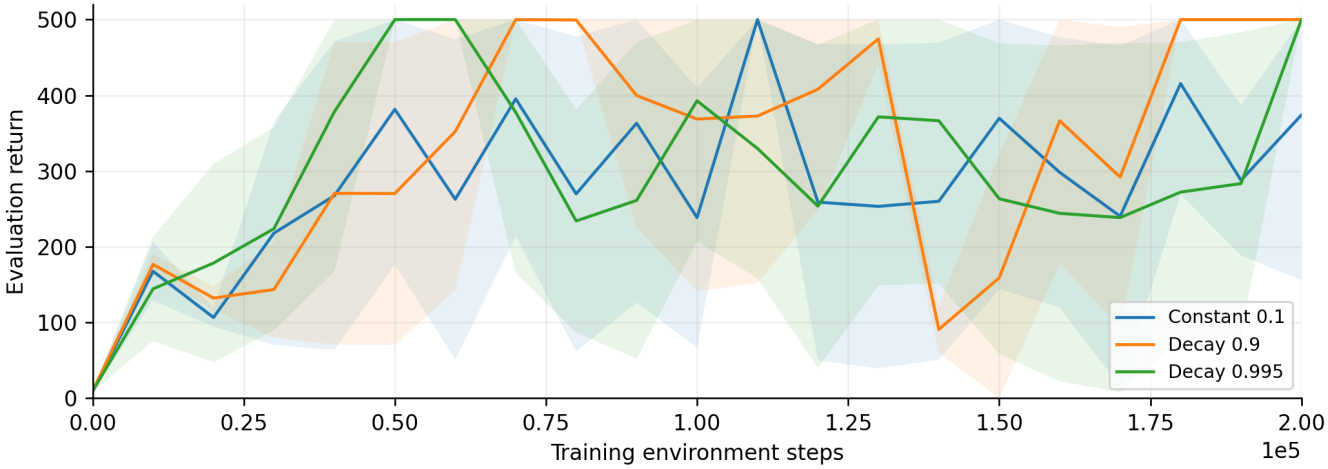


Figure 3. Exploration-schedule comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
Constant 0.1	374.1 ± 218.1	359.1 ± 32.3	80,000 (2/3)
Decay 0.9, floor 0.02	500.0 ± 0.0	500.0 ± 0.0	90,000 (3/3)
Decay 0.995, floor 0.02	500.0 ± 0.0	351.9 ± 82.1	60,000 (3/3)

Discussion

How do the schedules affect exploration and learning?

Constant epsilon maintains a 10% random-action probability. The 0.9 schedule reaches its floor at 399-470 steps; the 0.995 schedule reaches it at 114,641-143,081 steps. The fast schedule therefore becomes mostly greedy before the 1,000-step learning start, yet gives the best last-three return here.

Does keeping epsilon higher for longer improve performance or unnecessarily slow learning?

Longer exploration does not improve late performance in these runs: the last-three means are 500.0 for fast decay, 351.9 for slow decay and 359.1 for constant epsilon. Slow decay confirms the near-maximum threshold earlier, however, so this is not a universal speed ranking.

What happens if epsilon is too small or decays too quickly?

In general, too little exploration can miss useful actions. That failure is not demonstrated here: fast decay performs well on CartPole; state-space coverage was not measured, so the curves alone cannot explain why.

Task 2 - Question 3: Replay capacity and batch size

Question

Compare replay capacities 1,000, 10,000 and 100,000, and batch sizes 32 and 128. All six combinations use the same other settings.

Submit

Submit evaluation returns versus environment steps for all cases. In the table, the setting is replay capacity / batch size.

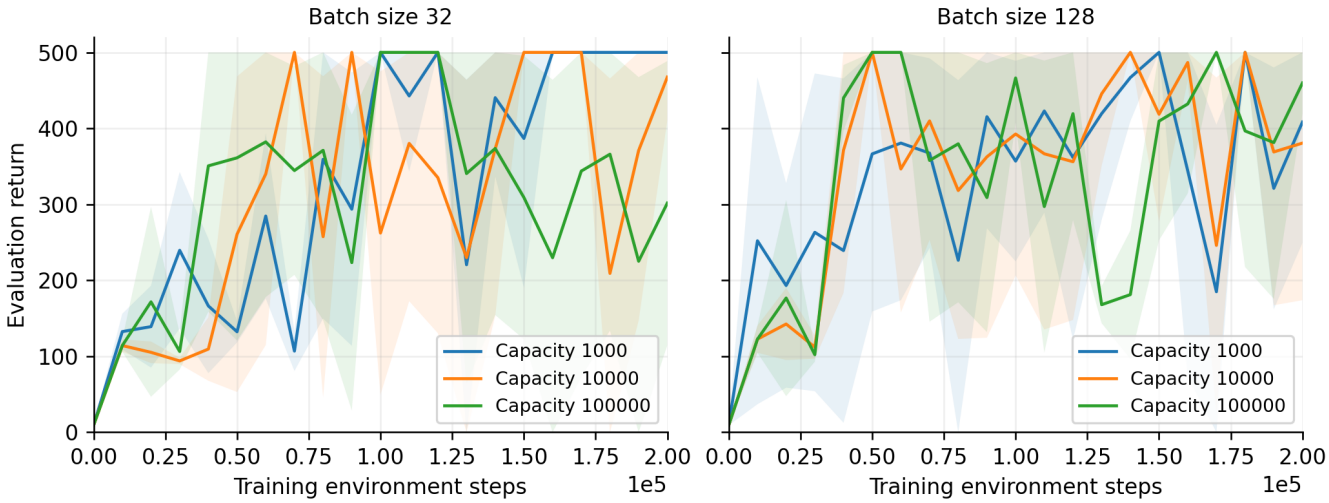


Figure 4. Replay-capacity and batch-size comparison.

Setting	Final return	Last-three return	Steps to threshold (seeds)
1000 / 32	500.0 ± 0.0	500.0 ± 0.0	120,000 (3/3)
1000 / 128	408.3 ± 158.9	409.8 ± 71.8	130,000 (3/3)
10000 / 32	467.5 ± 56.3	349.1 ± 162.6	110,000 (3/3)
10000 / 128	380.6 ± 206.8	416.5 ± 63.7	100,000 (3/3)
100000 / 32	301.6 ± 186.9	297.5 ± 60.0	120,000 (3/3)
100000 / 128	459.7 ± 40.4	412.5 ± 67.6	180,000 (3/3)

Discussion

How does memory size change curve stability and smoothness?

The effect depends on batch size. With batch 32, capacity 1,000 is steadier late in training than 100,000 (temporal SD 0.0 versus 185.7); with batch 128, increasing capacity reduces late temporal SD from 166.2 to 113.7.

How does batch size affect convergence speed and final performance?

Batch 128 does not always improve performance. At capacity 1,000 it lowers last-three return from 500.0 to 409.8; at 10,000 it raises it from 349.1 to 416.5 and confirms the threshold 10,000 steps earlier; at 100,000 it raises return but confirms 60,000 steps later.

Which combination gives the best CartPole performance in these experiments?

Capacity 1,000 with batch 32 is best by last-three return: 500.0 ± 0.0. This is the clearest late-performance result within the tested budget, not a claim that it is best for every task or performance measure.